НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ЮРИДИЧНА АКАДЕМІЯ»

Факультет кібербезпеки · Кафедра інформаційних технологій

Лабораторна робота 7

Тестування архітектури

Варіант 12 · Управління автопарком та маршрутами

Черниченко Владислав · Коцюба Максим

FleetFlow · 27 жовтня 2026 року

Мета та отриманий результат

Підготовлено 68 автоматизованих перевірок, з яких 67 виконуються у функціональному етапі та одна - в окремому HTTP performance gate. Покриття виконуваних рядків двох поточних доменних модулів становить 100%. Заборонений імпорт зупиняє той самий локальний конвеєр, який налаштовано для GitHub Actions.

Базові 26 тестів C:\lab5 пройшли до змін за 2.81 с. Після додавання безпеки тести потребують Bearer-облікових даних; попередні твердження про бізнес-результат не переписувалися. Репозиторій попередніх робіт: https://github.com/VladyslavCh412/labs; новий хмарний запуск не виконувався.

1 Аудит наявних і доповнених тестів

Рівень	Кількість	Час кейсів с	Залежності
Модульні	27	0.032	Без SQLite і HTTP-сервера
Інтеграційні	30	3.864	Тимчасовий SQLite / in-process ASGI
Контрактні	7	0.11	Окремі очікування; provider SQLite або MockTransport
Fitness	4	5.407	AST та окремий реальний HTTP-процес

Час - сума атрибутів testcase.time у verification/test_results.xml та fitness_results.xml. Це не wall-clock тривалість усіх процесів і не порівняння швидкості різних машин. Подробиці кожного кейсу - verification/test_audit.json.

Розподіл не містить сотень повільних наскрізних UI-перевірок, проте інтеграційних тестів трохи більше за модульні: це успадковано від лабораторних про SQLite, outbox та міграції. 21 новий доменний кейс посилює нижній рівень. Один socket HTTP gate перевіряє саме мережеву затримку; live_smoke.py окремо перевіряє два критичних бізнес-потоки.

2 Бізнес правила ЛР2 і модульні тести

Правило	Тест у tests/test_domain.py	Перевірка
Vehicle id, номер і пробіг	test_vehicle_identity_plate_mileage	3 варіанти; без сховища
Початок рейсу лише available	test_only_available_vehicle_can_start	in_trip та maintenance відхиляються
Заборона ТО під час рейсу	test_maintenance_cannot_interrupt_trip	Стан залишається IN_TRIP
Допуск водія	test_driver_permission	Усі 4 комбінації active/license_valid
Не менше двох непорожніх точок	test_route_requires_two_nonempty_stops	Порожній, один пункт, пробіл
Життєвий цикл і пробіг	test_trip_lifecycle_and_mileage	planned → active → completed; mileage +25
Некоректні переходи	test_trip_rejects_invalid_transitions_without_side_effects	Раннє complete, повторний start, недопущений водій, від'ємна відстань
Обов'язкові ID рейсу	test_trip_requires_all_identifiers	Усі 4 пропущені ID
Завершення лише активного рейсу	test_vehicle_finish_requires_active_trip_and_nonnegative_distance	Перехід до ТО та відмова finish

Під час аудиту виявлено, що після декомпозиції частина методів begin_trip, finish_trip, start_maintenance, activate і complete з ЛР2 вже не була присутня в поточних доменних файлах ЛР5. Їх відновлено як незалежну доменну логіку та покрито тестами. Вони не створюють прямої залежності між сервісами.

Поточний HTTP сценарій залишається плануванням рейсу. Окремі HTTP start/complete та розподілене резервування ресурсів не додавалися. mileage_km у поточній доменній моделі не має окремої міграції збереження в Fleet; це не заявляється як готовий production use case. Повне покриття рядків не означає повне покриття системи.

3 Сховище та контракт споживач постачальник

Інтеграційні тести ЛР5 перевіряють чотири етапи expand/contract, dual-write, сумісне читання, повторне застосування міграції, кеш TTL та інвалідацію. Повторна й неупорядкована доставка подій перевіряє event_id та version у незалежній БД Trips.

Контракт очікувань Trips знаходиться в contracts/fleet_consumer.json: обов'язкові поля Vehicle і Driver, типи, статуси та timeout 0.35 с. Контракт не генерується з відповідей provider під час тесту: він зафіксований окремо.

Provider-тест створює справжній Fleet із тимчасовим SQLite і перевіряє його відповіді JSON Schema. Consumer-тест не запускає Fleet: він підставляє MockTransport, звіряє приклад зі схемою та викликає FleetClient. Окремий негативний тест видаляє поле plate і підтверджує ValidationError; помилки 404/409/503 та timeout мають очікуване відображення.

Машиночитні описи API з новими параметрами сторінки та Bearer: contracts/fleet.secured.openapi.json і trips.secured.openapi.json. Історичні YAML ЛР3 збережено як попередню специфікацію.

4 Архітектурні fitness функції

Функція	Правило та поріг	Механізм
Статична 1	0 заборонених імпортів у *_service/domain.py	AST; дозволено лише dataclasses, enum, typing, collections, datetime, decimal, __future__.
Статична 2	0 циклів між модулями поточних сервісів і кореневими .py	AST-граф локальних імпортів; обхід DFS з активним стеком.
Динамічна	p95 ≤ 500 мс, 0 помилок, ≥100 запитів при 100 користувачах	Окремий Uvicorn, 5000 рейсів, реальний loopback HTTP, 3-секундний closed-loop прогін.

Фактичний незалежний gate: p50 365.6 мс, p95 405.5 мс, 850 запитів, 0 помилок. Дані - verification/fitness_measurement.json. Поріг узято з lab1.docx, а не підлаштовано під результат.

Статичний аналіз не відстежує динамічний importlib, файловий SQL-доступ, ORM-конфігурацію або імпорти всередині сторонніх пакетів. Ці обмеження зазначені явно; перевірка відсутності циклів не є формальним доказом усієї архітектури.

5 Конвеєр складання та негативний експеримент

verification/pipeline.py виконує: (1) статичні правила, (2) усі функціональні тести з JUnit і coverage порогом 95% для двох domain.py, (3) окремий performance gate без інструментування coverage. Будь-який ненульовий код негайно зупиняє збірку. Відокремлення замірів усуває вплив вимірювання покриття на HTTP latency.

GitHub Actions налаштований для push, pull_request і workflow_dispatch. Workflow потрібно розмістити в корені репозиторію .github/workflows/architecture.yml; він запускає матрицю lab7 та lab8 на Python 3.13. Повний комплект GITHUB_READY містить правильне розташування. Файл усередині окремого lab7 сам по собі не активує Actions.

verification/negative_pipeline.py створює тимчасову копію джерел, додає import sqlite3 до fleet_service/domain.py і запускає справжній verification/pipeline.py --static-only. Результат: ARCHITECTURE FAILED; forbidden domain import sqlite3; exit 1. SHA/байти оригінального файла порівнюються до й після; оригінал не змінюється.

Повний журнал - verification/negative_pipeline.log; структурований результат - negative_pipeline.json. Це підтверджене локальне падіння того самого gate, а не вигаданий скриншот хмарного запуску. Хмарний CI виконається після розміщення коду у GitHub.

6 Відтворення та висновки

Команди після встановлення requirements-dev.txt: python verification/pipeline.py; python verification/negative_pipeline.py; python verification/live_smoke.py. Тести не використовують робочі data/*.sqlite. Тимчасові SQLite-з’єднання закриваються явно для Windows.

Захищено чистоту домену, відсутність статичних циклів, сумісність Fleet/Trips, міграції, ідемпотентність, доступ і заданий локальний p95. Не покрито реальний браузер усіма сценаріями, fault injection на 10 хвилин, GPS, резервне відновлення, аудит зміни ролей та production performance.

Ролі для підготовки: Владислав - fitness і конвеєр; Максим - контракт, аудит і бізнес-тести. Кожен має пояснити код і обмеження на захисті. Джерела: методичка ЛР7, lab1.docx–lab5.docx, документація pytest та GitHub Actions.